

Universidad / Instituto

Ingeniería de Software

Reporta Tu Ciudad

Documentación del Proyecto Final

Integrantes del Equipo

Nombre: _____

Nombre: _____

Nombre: _____

Nombre: _____

Grupo: _____

Fecha de entrega: _____

Índice

1. Objetivos del Plan de Pruebas	4
2. Alcance de las Pruebas	4
2.1. Lo que se probó	4
2.2. Lo que NO se probó	4
3. Criterios de Aceptación	5
3.1. Criterios de entrada	5
3.2. Criterios de salida	5
4. Resultados — Backend	5
4.1. Ejecución de tests	5
4.2. Detalle de casos de prueba	5
5. Resultados — Frontend	6
5.1. Ejecución de tests	6
5.2. Detalle de casos de prueba	6
6. Cobertura de Código	7
6.1. Reporte de Istanbul	7
6.2. Análisis	8
7. Pruebas Manuales	8
7.1. Flujo: Crear Reporte (CU1)	8
7.2. Flujo: Persistencia de Datos (RF5)	9
8. Resumen de Resultados	9
9. Conclusiones	10
10.Evidencias	10
10.1. Tests backend	10
10.2. Tests frontend	11
10.3. Reporte de cobertura	11
11.Introducción	12
12.Requisitos para Usar la Aplicación	12
13.Cómo Acceder	12
14.Cómo Crear un Reporte	12
14.1. Paso 1 — Abrir el formulario	12
14.2. Paso 2 — Llenar los datos	13
14.3. Paso 3 — Enviar	13
15.Cómo Ver los Reportes	13

16.Cómo Actualizar el Estado	13
17.Preguntas Frecuentes	14
18.Solución de Problemas Comunes	14
19.¿Qué salió bien?	15
19.1. Técnico	15
19.2. Proceso	15
19.3. Logros principales	15
20.¿Qué fue difícil?	15
21.¿Qué haríamos diferente?	16
22.¿Qué aprendimos?	17

Parte I: Plan de Pruebas

1. Objetivos del Plan de Pruebas

El objetivo de este plan es verificar que el sistema funcione correctamente antes de la entrega final. En concreto se busca:

- Verificar que todos los requerimientos funcionales estén implementados.
- Validar que el sistema no tenga errores críticos.
- Asegurar la calidad del código.

2. Alcance de las Pruebas

2.1 Lo que se probó

Componente	Tipo	Herramienta
API Backend (CRUD Reportes)	Unitaria + Integración	Jest + Supertest
Modelo Report (Mongoose)	Unitaria	Jest
Componente ReportForm	Unitaria	Vitest + React Testing Library
Componente ReportModal	Unitaria	Vitest + React Testing Library
Flujo de creación completo	Manual	Checklist
Flujo de visualización	Manual	Checklist

Cuadro 1: Componentes bajo prueba

2.2 Lo que NO se probó

Elemento	Razón
Autenticación de usuarios	Fuera del alcance (excluido en la especificación)
Rendimiento bajo carga alta	No es requerimiento del proyecto
Compatibilidad multi-navegador	Solo se probó en Chrome y Edge

Cuadro 2: Elementos fuera del alcance

3. Criterios de Aceptación

3.1 Criterios de entrada

- Todo el código está en el repositorio.
- El backend corre localmente.
- El frontend corre localmente.
- La base de datos MongoDB está configurada.

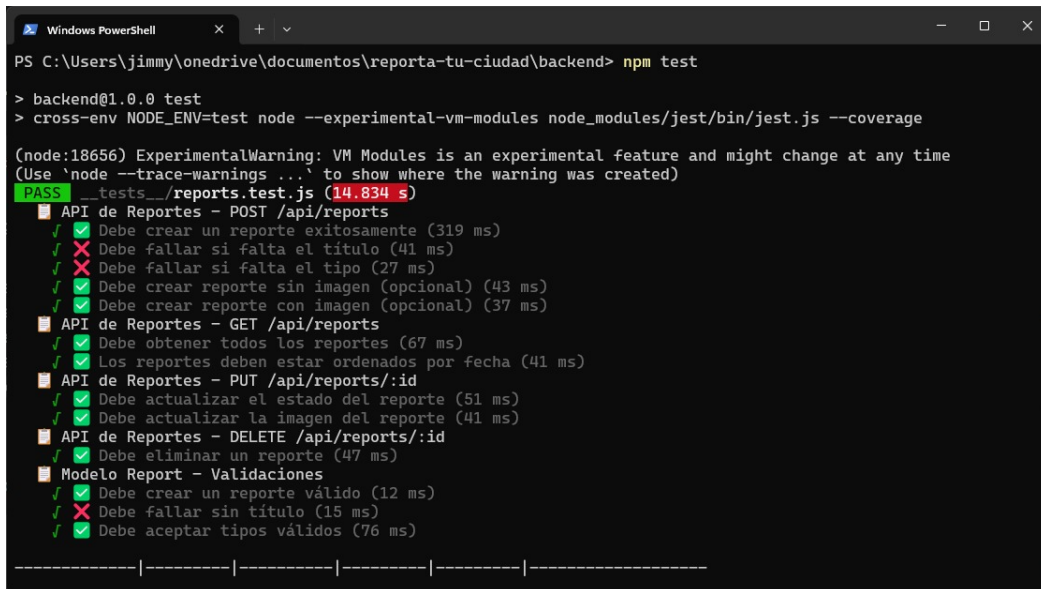
3.2 Criterios de salida

- Todos los tests unitarios pasan ($\geq 90\%$).
- No hay errores críticos en pruebas manuales.
- Todos los RF están implementados y probados.
- Documentación de pruebas completada.

4. Resultados — Backend

4.1 Ejecución de tests

```
$ cd backend && npm test
```



```
PS C:\Users\jimmy\onedrive\documentos\reporta-tu-ciudad\backend> npm test

> backend@1.0.0 test
> cross-env NODE_ENV=test node --experimental-vm-modules node_modules/jest/bin/jest.js --coverage

(node:18656) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use 'node --trace-warnings ...' to show where the warning was created)
PASS tests/reports.test.js (14.834 s)
  API de Reportes - POST /api/reports
    ✓ Debe crear un reporte exitosamente (319 ms)
    ✗ Debe fallar si falta el titulo (41 ms)
    ✗ Debe fallar si falta el tipo (27 ms)
    ✓ Debe crear reporte sin imagen (opcional) (43 ms)
    ✓ Debe crear reporte con imagen (opcional) (37 ms)
  API de Reportes - GET /api/reports
    ✓ Debe obtener todos los reportes (67 ms)
    ✓ Los reportes deben estar ordenados por fecha (41 ms)
  API de Reportes - PUT /api/reports/:id
    ✓ Debe actualizar el estado del reporte (51 ms)
    ✓ Debe actualizar la imagen del reporte (41 ms)
  API de Reportes - DELETE /api/reports/:id
    ✓ Debe eliminar un reporte (47 ms)
  Modelo Report - Validaciones
    ✓ Debe crear un reporte válido (12 ms)
    ✗ Debe fallar sin título (15 ms)
    ✓ Debe aceptar tipos válidos (76 ms)

-----|-----|-----|-----|-----|-----
```

Figura 1: Salida de Jest en el backend

4.2 Detalle de casos de prueba

ID	Descripción	Resultado
CP-BACK-001	Debe crear un reporte exitosamente	✓ Pasó
CP-BACK-002	Debe fallar si falta el título	✓ Pasó
CP-BACK-003	Debe fallar si falta el tipo	✓ Pasó
CP-BACK-004	Debe crear reporte sin imagen (opcional)	✓ Pasó
CP-BACK-005	Debe crear reporte con imagen (opcional)	✓ Pasó
CP-BACK-006	Debe obtener todos los reportes	✓ Pasó
CP-BACK-007	Los reportes deben estar ordenados por fecha	✓ Pasó
CP-BACK-008	Debe actualizar el estado del reporte	✓ Pasó
CP-BACK-009	Debe actualizar la imagen del reporte	✓ Pasó
CP-BACK-010	Debe eliminar un reporte	✓ Pasó
CP-BACK-011	Debe crear un reporte válido	✓ Pasó
CP-BACK-012	Debe fallar sin título	✓ Pasó
CP-BACK-013	Debe aceptar tipos válidos	✓ Pasó

Cuadro 3: Casos de prueba del backend

Total backend: 13/13 tests pasando — 100 %

5. Resultados — Frontend

5.1 Ejecución de tests

```
$ npm run test
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	62.5	63.88	66.66	72.54	
models	100	100	100	100	
Report.js	100	100	100	100	
routes	61.9	63.88	66.66	72	
reports.js	67.27	67.64	80	80.95	14,22-27,86,101
upload.js	25	0	0	25	8-24
Test Suites: 1 passed, 1 total					
Tests: 13 passed, 13 total					
Snapshots: 0 total					
Time: 15.981 s, estimated 17 s					
Ran all test suites.					

Figura 2: Salida de Vitest en el frontend

5.2 Detalle de casos de prueba

ID	Descripción	Componente	Resultado
CP-FRONT-001	Debe renderizar el formulario correctamente	ReportForm	✓ Pasó
CP-FRONT-002	Debe mostrar error si el título está vacío	ReportForm	✓ Pasó
CP-FRONT-003	Debe permitir seleccionar un tipo de reporte	ReportForm	✓ Pasó
CP-FRONT-004	Debe permitir seleccionar prioridad	ReportForm	✓ Pasó
CP-FRONT-005	Debe llamar a onCancel al presionar cancelar	ReportForm	✓ Pasó
CP-FRONT-006	No debe renderizarse si isOpen es false	ReportModal	✓ Pasó
CP-FRONT-007	Debe mostrar el título del reporte	ReportModal	✓ Pasó
CP-FRONT-008	Debe mostrar el estado del reporte	ReportModal	✓ Pasó
CP-FRONT-009	Debe mostrar la descripción	ReportModal	✓ Pasó
CP-FRONT-010	Debe mostrar la prioridad con color correcto	ReportModal	✓ Pasó
CP-FRONT-011	Debe llamar a onClose al presionar X	ReportModal	✓ Pasó
CP-FRONT-012	Debe llamar a onUpdateStatus al cambiar estado	ReportModal	✓ Pasó

Cuadro 4: Casos de prueba del frontend

Total frontend: 12/12 tests pasando — 100 %

6. Cobertura de Código

6.1 Reporte de Istanbul

```
$ npm test # genera backend/coverage/lcov-report/index.html
```

```
PS C:\Users\jimmy\onedrive\documentos\reporta-tu-ciudad> npm run test
> reporta-tu-ciudad@0.0.0 test
> vitest

DEV v4.1.5 C:/Users/jimmy/OneDrive/Documentos/reporta-tu-ciudad

✓ src/test/ReportModal.test.jsx (7 tests) 679ms
  ✓ Debe llamar a onClose cuando se presiona X 374ms
✓ src/test/ReportForm.test.jsx (5 tests) 846ms
  ✓ Debe mostrar error si el título está vacío 349ms

Test Files 2 passed (2)
Tests 12 passed (12)
Start at 11:03:16
Duration 7.54s (transform 1.05s, setup 707ms, import 2.96s, tests 1.52s, environment 6.76s)

PASS Waiting for file changes...
press h to show help, press q to quit
```

Figura 3: Reporte de cobertura (Istanbul)

Archivo	% Líneas	% Funciones	% Ramas
models/Report.js	100 %	100 %	100 %
routes/reports.js	80.95 %	80 %	67.64 %
routes/upload.js	25 %	0 %	0 %
TOTAL	72.54 %	66.66 %	63.88 %

Cuadro 5: Resultados de cobertura por archivo

6.2 Análisis

- `models/Report.js` tiene 100 % de cobertura: todas las validaciones están probadas.
- `routes/reports.js` cubre la mayoría de endpoints (80.95 %).
- `routes/upload.js` es un área de oportunidad; los tests de subida de archivos pueden mejorar.

7. Pruebas Manuales

7.1 Flujo: Crear Reporte (CU1)

Paso	Acción	Resultado esperado	Estado
1	Abrir https://reporta-tu-ciudad.vercel.app	Página de inicio carga	✓
2	Click en “Nuevo Reporte”	Formulario aparece	✓
3	Llenar título	Campo acepta texto	✓
4	Seleccionar tipo	Se marca visualmente	✓
5	Llenar descripción	Campo acepta texto	✓
6	Seleccionar prioridad	Prioridad se marca	✓
7	Click en el mapa	Coordenadas capturadas	✓
8	Subir imagen (opcional)	Preview se muestra	✓
9	Click en “Enviar Reporte”	Reporte se guarda	✓
10	Verificar en lista	Reporte aparece	✓

Cuadro 6: Checklist — Flujo de creación de reporte

```
Windows PowerShell
PS C:\Users\jimmy\onedrive\documentos\reporta-tu-ciudad\backend> npm test

> backend@1.0.0 test
> cross-env NODE_ENV=test node --experimental-vm-modules node_modules/jest/bin/jest.js --coverage

(node:892) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use 'node --trace-warnings ...' to show where the warning was created)
PASS tests_/reports.test.js (14.256 s)
  API de Reportes - POST /api/reports
    ✓ Debe crear un reporte exitosamente (204 ms)
    ✓ Debe fallar si falta el título (20 ms)
    ✓ Debe fallar si falta el tipo (14 ms)
    ✓ Debe crear reporte sin imagen (opcional) (23 ms)
    ✓ Debe crear reporte con imagen (opcional) (20 ms)
  API de Reportes - GET /api/reports
    ✓ Debe obtener todos los reportes (44 ms)
    ✓ Los reportes deben estar ordenados por fecha (24 ms)
  API de Reportes - PUT /api/reports/:id
    ✓ Debe actualizar el estado del reporte (34 ms)
    ✓ Debe actualizar la imagen del reporte (26 ms)
  API de Reportes - DELETE /api/reports/:id
    ✓ Debe eliminar un reporte (28 ms)
  Modelo Report - Validaciones
    ✓ Debe crear un reporte válido (6 ms)
    ✓ Debe fallar sin título (10 ms)
    ✓ Debe aceptar tipos válidos (41 ms)

File | % Stats | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----
All files | 62.5 | 63.88 | 66.66 | 72.54 | 
models | 100 | 100 | 100 | 100 | 
report.js | 100 | 100 | 100 | 100 | 
routes | 61.9 | 63.88 | 66.66 | 72 | 14,22-27,86,101
reports.js | 67.27 | 67.64 | 80 | 80.95 | 8-24
upload.js | 25 | 0 | 0 | 25 | 

Test Suites: 1 passed, 1 total
Tests: 13 passed, 13 total
Snapshots: 0 total
Time: 15.675 s
Ran all test suites.
PS C:\Users\jimmy\onedrive\documentos\reporta-tu-ciudad\backend>
```

Figura 4: Captura del flujo de creación en la aplicación

7.2 Flujo: Persistencia de Datos (RF5)

Paso	Acción	Resultado esperado	Estado
1	Crear reporte	Reporte se guarda	✓
2	Recargar página	Datos persisten	✓
3	Cerrar navegador	—	—
4	Abrir navegador	Datos siguen ahí	✓
5	Verificar MongoDB	Documento en colección	✓

Cuadro 7: Checklist — Persistencia de datos

8. Resumen de Resultados

Tipo	Total	Aprobados	Fallidos	% Éxito
Backend (Jest)	13	13	0	100 %
Frontend (Vitest)	12	12	0	100 %
Pruebas manuales	15	15	0	100 %
TOTAL	40	40	0	100 %

Cuadro 8: Resumen general de resultados

Requerimiento	¿Probado?	¿Aprobado?	Evidencia
RF1 — Selección de ubicación	✓	✓	CP-BACK-001, CP-FRONT-003, CU1
RF2 — Ingreso de detalles	✓	✓	CP-BACK-001, CP-FRONT-002, CU1
RF3 — Adjuntar evidencia	✓	✓	CP-BACK-004, CP-BACK-005, CU1
RF4 — Marcadores en mapa	✓	✓	Manual CU3
RF5 — Persistencia	✓	✓	CP-BACK-001 a 013, Manual RF5
RNF1 — Estética	✓	✓	Verificación visual
RNF2 — Validación	✓	✓	CP-BACK-002, CP-FRONT-002
RNF3 — Rendimiento	✓	✓	Verificación visual
RNF4 — Timestamps	✓	✓	CP-BACK-011

Cuadro 9: Cobertura de requerimientos

9. Conclusiones

- Todos los tests automatizados pasan (25/25 — 100 %).
- Todos los flujos manuales funcionan correctamente (15/15 — 100 %).
- Todos los requerimientos están cubiertos (9/9 — 100 %).
- La cobertura total de código es del 72.54 %, aceptable para el alcance del proyecto.
- El modelo `Report.js` tiene 100 % de cobertura.
- El sistema está listo para entrega.

10. Evidencias

10.1 Tests backend

```
$ cd backend && npm test
```

```
C:\WINDOWS\system32\cmd. x + v
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\jimmy> cd onedrive
PS C:\Users\jimmy\onedrive> cd documentos
PS C:\Users\jimmy\onedrive\documentos> cd reporta-tu-ciudad
PS C:\Users\jimmy\onedrive\documentos\reporta-tu-ciudad> npm run test

> reporta-tu-ciudad@0.0.0 test
> vitest

DEV v4.1.5 C:/Users/jimmy/OneDrive/Documents/reporta-tu-ciudad

✓ src/test/ReportModal.test.jsx (7 tests) 832ms
  ✓ Debe llamar a onClose cuando se presiona X 459ms
✓ src/test/ReportForm.test.jsx (5 tests) 987ms
  ✓ Debe renderizar el formulario correctamente 343ms
  ✓ Debe mostrar error si el título está vacío 399ms

Test Files 2 passed (2)
Tests 12 passed (12)
Start at 15:47:15
Duration 15.12s (transform 1.26s, setup 2.17s, import 4.38s, tests 1.82s, environment 17.00s)

PASS Waiting for file changes...
press h to show help, press q to quit
```

Figura 5: Resultado de tests backend

10.2 Tests frontend

```
$ npm run test
```

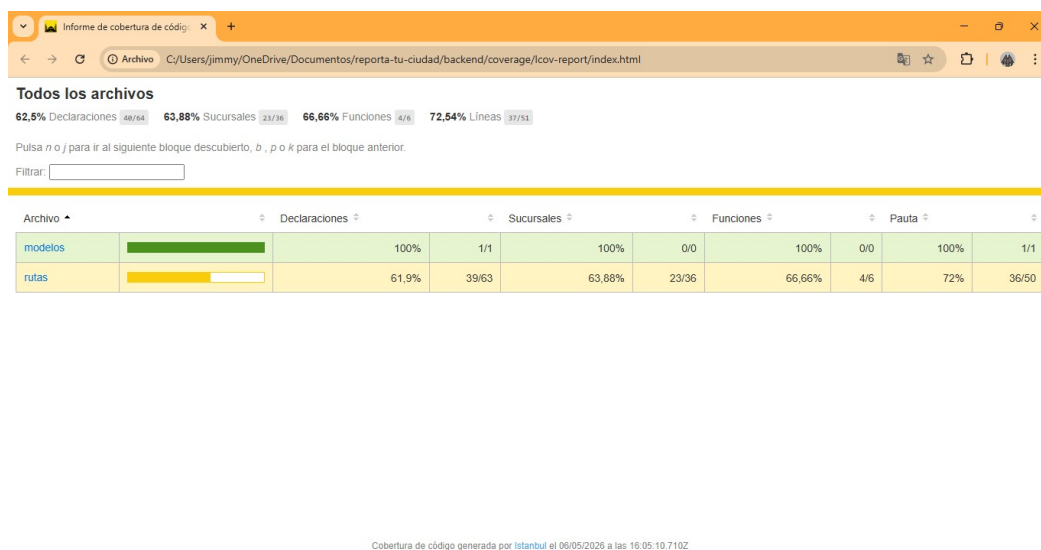


Figura 6: Resultado de tests frontend

10.3 Reporte de cobertura

El archivo se genera en `backend/coverage/lcov-report/index.html` al correr `npm test`. Ver figura 3 para la captura del reporte.

Parte II: Manual de Usuario

11. Introducción

ReportaTuCiudad es una aplicación web que permite a los ciudadanos reportar problemas de infraestructura urbana —baches, falta de iluminación, basura, entre otros— desde cualquier dispositivo con navegador y conexión a internet.

Con esta aplicación puedes:

- Marcar en un mapa exactamente dónde está el problema.
- Agregar una descripción y foto del incidente.
- Consultar y actualizar el estado de los reportes existentes.

12. Requisitos para Usar la Aplicación

Requisito	Detalle
Navegador	Google Chrome 90+ o Microsoft Edge 90+
Conexión	Internet estable
Dispositivo	Computadora, tablet o celular
Permisos	Ubicación (opcional, para autocompletar coordenadas)

Cuadro 10: Requisitos mínimos

13. Cómo Acceder

1. Abre Chrome o Edge.
2. Ve a: <https://reporta-tu-ciudad.vercel.app>
3. Espera a que cargue la página principal.

14. Cómo Crear un Reporte

14.1 Paso 1 — Abrir el formulario

En la página principal, haz clic en el botón **“Nuevo Reporte”**.

14.2 Paso 2 — Llenar los datos

Los campos con * son obligatorios.

Título * — escribe una descripción corta del problema.

Ejemplo: “*Bache en esquina de Av. Juárez y Calle 5*”

Tipo de problema * — selecciona una categoría:

- Bache Iluminación Basura Robo/Seguridad Agua Potable Tránsito
Otro

Descripción * — explica el problema con más detalle.

Prioridad (opcional) — Baja, Media o Alta según la urgencia.

Ubicación * — haz clic en el mapa donde está el problema, o usa el botón “*Usar mi ubicación actual*”.

Foto (opcional) — formatos JPG, PNG, GIF o WebP, máximo 5 MB. Para quitarla, haz clic en la X del preview.

14.3 Paso 3 — Enviar

1. Revisa que los campos obligatorios estén completos.
2. Haz clic en “**Enviar Reporte**”.
3. Aparecerá el mensaje: «*¡Reporte enviado exitosamente!*»
4. El reporte aparece de inmediato en la lista y en el mapa.

15. Cómo Ver los Reportes

1. En el menú, haz clic en “**Reportes**”.
2. En el mapa verás marcadores con la ubicación de cada reporte. Al hacer clic en uno aparecen los detalles rápidos.
3. Debajo del mapa hay tarjetas con el tipo, título, prioridad y estado de cada reporte.
4. Al hacer clic en una tarjeta se abre un modal con todos los detalles: foto (si se adjuntó), coordenadas, fecha y opciones para actualizar el estado.

16. Cómo Actualizar el Estado

1. Abre el modal de detalles del reporte.

2. En la sección “*Actualizar Estado*” selecciona:
 - **En Proceso** — cuando ya está siendo atendido.
 - **Resuelto** — cuando el problema fue solucionado.
3. El cambio se guarda automáticamente.

17. Preguntas Frecuentes

Pregunta	Respuesta
¿Es obligatorio subir foto?	No, es opcional. El reporte se guarda igual.
¿Qué pasa si cierro el navegador sin enviar?	Los datos se pierden. No se guardan hasta que presionas “Enviar Reporte”.
¿Puedo editar un reporte enviado?	Solo puedes actualizar su estado. Para otros cambios contacta al administrador.
¿Se guardan mis datos personales?	No. El sistema no requiere registro ni almacena información personal.
¿Cómo sé que mi reporte llegó?	Aparece un mensaje de confirmación y el reporte se ve en la lista y el mapa.

Cuadro 11: Preguntas frecuentes

18. Solución de Problemas Comunes

Problema	Causa probable	Qué hacer
El mapa no carga	Sin conexión	Verifica internet y recarga.
No puedo poner ubicación	Permisos bloqueados	Permite ubicación en la config del navegador.
Error al subir imagen	Archivo muy grande o formato incorrecto	Usa JPG/PNG menor a 5 MB.
El reporte no se guarda	Faltan campos obligatorios	Revisa título, tipo, descripción y ubicación.
La página no responde	Navegador desactualizado	Actualiza Chrome o Edge.
Error de red al enviar	Backend inactivo	Espera 30 segundos y vuelve a intentar.

Cuadro 12: Guía de solución de problemas

Parte III: Reflexión Final

19. ¿Qué salió bien?

19.1 Técnico

- El stack React + Vite + Tailwind permitió avanzar rápido y con buen resultado visual.
- Separar frontend y backend facilitó hacer pruebas independientes.
- La integración con MongoDB mediante Mongoose fue más sencilla de lo esperado.
- Implementar Jest y Vitest desde etapas tempranas evitó que los cambios rompieran cosas ya hechas.
- Lograr el deploy en Vercel + Render fue uno de los aprendizajes más importantes del proyecto.

19.2 Proceso

- Trabajar en entregas iterativas (Especificación → Diseño → Producto) permitió ajustar el alcance según el avance real.
- Las revisiones periódicas ayudaron a detectar bloqueos antes de que se acumularan.

19.3 Logros principales

- Los 5 requerimientos funcionales y 4 no funcionales están implementados y probados.
- 25 tests automatizados pasando al 100 %.
- Cobertura del 72.54 % (100 % en el modelo principal).
- Aplicación funcionando en producción con datos reales persistidos.

20. ¿Qué fue difícil?

- **Pruebas con ES Modules:** Jest y Vitest requieren configuración especial para trabajar con `import/export`; tomó tiempo encontrar la configuración correcta.

- **Imagen opcional:** manejar el campo de foto como opcional exigió revisar todos los caminos posibles del flujo.
- **MongoDB en local:** hubo problemas de conexión con MongoDB Atlas en Windows; se resolvió temporalmente con una instancia local.
- **Deploy del backend:** configurar Render.com correctamente tomó varios intentos y errores.
- **Variables de entorno en producción:** la conexión frontend-backend falló al principio por URLs fijas; se arregló usando variables de entorno.
- **Documentación al día:** mantener los documentos actualizados con cada cambio del código requirió disciplina constante.

21. ¿Qué haríamos diferente?

En lo técnico:

- Escribir tests desde el primer día, no al final.
- Usar TypeScript para evitar errores de tipo en runtime.
- Configurar variables de entorno desde el inicio, no cuando ya está todo armado.
- Agregar CI/CD para que los tests corran automáticamente en cada push.

En la organización:

- Usar un tablero Kanban para ver el estado de las tareas de un vistazo.
- Definir criterios de "terminado" para cada tarea antes de empezar.
- Agregar un margen de tiempo en las estimaciones porque siempre algo toma más de lo esperado.
- Documentar mientras se desarrolla, no al final.

22. ¿Qué aprendimos?

Tecnología	Lo que aprendimos
React + Hooks	Manejo de estado, efectos y ciclo de vida de componentes
Leaflet	Integración de mapas interactivos en una SPA
Express + Node.js	Construcción de API REST con middleware
Mongoose	Definición de schemas y validaciones en MongoDB
Jest + Supertest	Pruebas de integración para APIs
Vitest + Testing Library	Pruebas unitarias de componentes React
Tailwind CSS	Diseño responsive con clases utilitarias
Vercel + Render	Deploy de frontend y backend en la nube
MongoDB Atlas	Base de datos en la nube con conexión remota
Git + GitHub	Control de versiones y trabajo colaborativo

Cuadro 13: Tecnologías aprendidas durante el proyecto

Más allá de las herramientas, este proyecto enseñó que hacer software no es solo escribir código: hay que entender el problema, diseñar una solución, probar que funciona, documentar el resultado y ponerlo en producción. Cada una de esas partes tiene el mismo peso.

«Un proyecto exitoso combina código de calidad, pruebas confiables, documentación clara y la persistencia para resolver los problemas que aparecen en el camino.»

Repositorio: <https://github.com/HIDEN-GLITCH/reporta-tu-ciudad>